

CS 550 Operating Systems

Spring 2025

Paging: smaller page tables

Problems of using page table

- Problem 1: using page table is slow → TLB.
- Problem 2: page tables are too big and thus consume too much memory.
 - How to solve this?
 - Bigger pages.
 - Combining paging and segments.
 - Multi-level page tables.
 - Inverted page tables

Solution 1: bigger pages

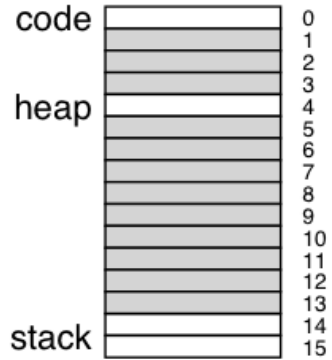
- 32 bit address, 4 KB pages, PTE is 4 bytes
 - 20 bit VPN, 12 bit offset
 - $2^{20} = 1\text{M}$ pages $\rightarrow$ 4MB per process
- 32 bit address, 16 KB pages
 - 18 bit VPN, 14 bit offset
 - $2^{18} = 256\text{K}$ pages $\rightarrow$ 1MB per process
- Problems?
 - Internal fragmentation
- Reality: 4KB or 8KB pages.

Solution 2: combining paging and segmentation

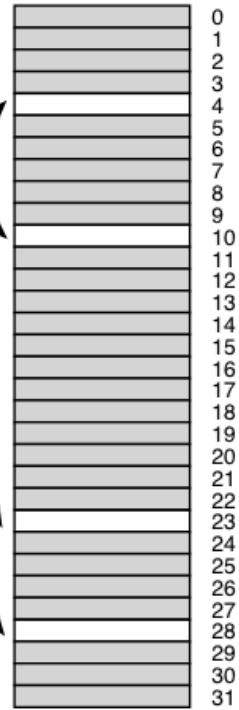
- Combining paging and segmentation to reduce the memory overhead of page tables.
- Why this might work?
 - Let's examine a typical linear page table in more detail.
 - 16KB address space, 1KB pages, 16 pages
 - How many bits in a VA? How many bits in VPN? How many bits in offset?
 - 14 bit virtual address: 4 bit VPN, 10 bit offset

16KB address space, 1KB pages, 16 pages

Virtual Address Space



Physical Memory



The linear page table (PT)

PFN	valid	prot	present	dirty
10	1	r-x	1	0
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
23	1	rw-	1	1
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
-	0	—	-	-
28	1	rw-	1	1
4	1	rw-	1	1

Every page needs an entry in the PT, even though many pages have not been loaded into memory yet (meaning the corresponding PTEs are invalid) → unnecessary big PT

How can we solve this problem by combining segmentation with paging?

The idea of combining segmentation and paging to form smaller page tables:

- Instead of having a single page table for the entire address space of the process, we can have one per logical segment.
- In the previous example, we might thus have three page tables, one for the code, heap, and stack parts of the address space.

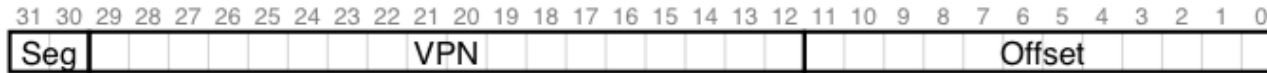
Example

- 32 bit address, 4KB pages, 4 segments in the address space (3 are used in the example: code, stack, and heap).

A VA:

SN	VPN	offset
----	-----	--------

How many bits in SN, VPN, offset? 2, 18, 12



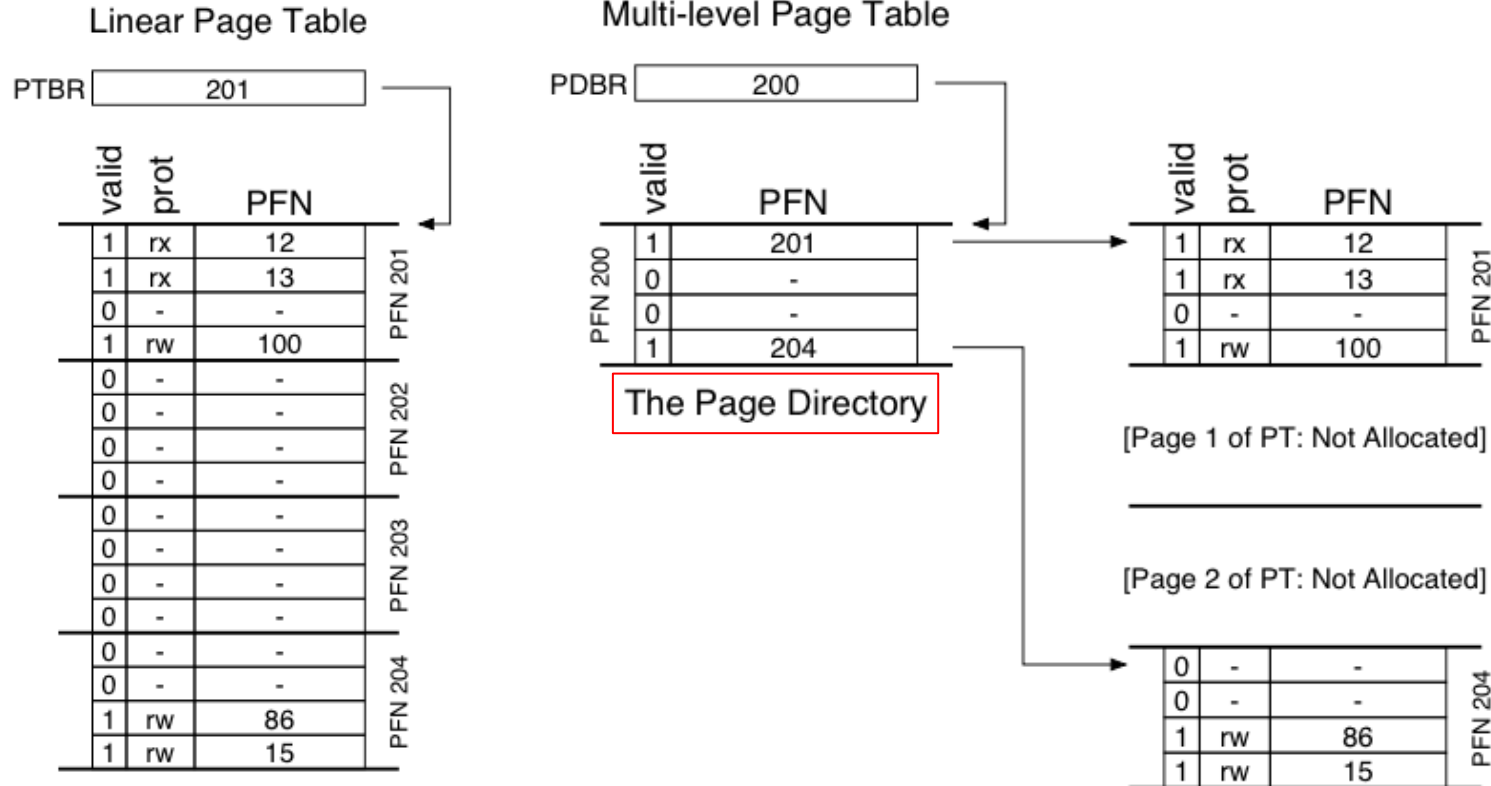
```
SN          = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN         = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

- Three base/bound registers for three page tables.
 - Selected using the SN bits.
 - Base register stores the physical address of the **linear page table for a segment**
 - Like the page table base register (PTBR) in linear paging
 - Bound register stores the **length of the page table**.
 - Recall that what content do the base/bound registers store in the segmentation approach?
- Problems?
 - Still use segmentation, which assumes a certain usage pattern of address space → not flexible
 - Page table can be of different size → free space management is complex

Solution 3: multi-level page tables

- The high-level idea → to get rid of those invalid regions in the page table instead of keeping them all in memory.
- The basic idea
 - chop up the page table into fixed-sized units
 - Usually the unit size is a page
 - if an entire unit of page-table entries (PTEs) is invalid, don't allocate that page of the page table at all.

Multi-level Page Tables



A multi-level paging example

- Basic setup
 - 128 bytes address space
 - Page size is 8 bytes.
 - PTE size is 4 bytes.
- Suppose linear paging is used
 - How many bits for VA, VPN, offset respectively?
 - VA: 7 bits ($2^7 = 128$)
 - VPN: 4 bits (there are $128 / 8 = 16 = 2^4$ pages)
 - Offset: 3 bits ($2^3 = 8$, which is the page size)
 - How many pages are needed to store a linear page table?
 - Since there are 2^4 pages, the linear page table has 2^4 PTEs.
 - Therefore, the size of a linear page table is $2^4 \times 4 = 64$ bytes, which occupy $64 / 8 = 2^6 / 2^3 = 2^3 = 8$ pages.

A multi-level paging example

Suppose the address space of a process P at a certain time looks like:

127	VP 15(used)
	VP 14 (free)
	VP 13 (free)
	VP 12 (free)
	VP 11 (free)
	VP 10 (free)
	VP 9 (free)
	VP 8 (free)
	VP 7 (free)
	VP 6 (free)
	VP 5 (free)
	VP 4 (free)
	VP 3 (free)
	VP 2 (free)
	VP 1 (free)
7	
0	Virtual page 0 (used)

Then the linear page table (LPT), which occupies 8 pages, looks like:

	PFN	Valid	
PTE 0	2	1	1 st page of the LPT (PFN 11)
PTE 1	-	0	
PTE 2	-	0	2 nd page of the LPT (PFN 12)
PTE 3	-	0	
PTE 4	-	0	3 rd page of the LPT (PFN 13)
PTE 5	-	0	
PTE 6	-	0	4 th page of the LPT (PFN 14)
PTE 7	-	0	
PTE 8	-	0	5 th page of the LPT (PFN 15)
PTE 9	-	0	
PTE 10	-	0	6 th page of the LPT (PFN 16)
PTE 11	-	0	
PTE 12	-	0	7 th page of the LPT (PFN 17)
PTE 13	-	0	
PTE 14	-	0	8 th page of the LPT (PFN 18)
PTE 15	3	1	

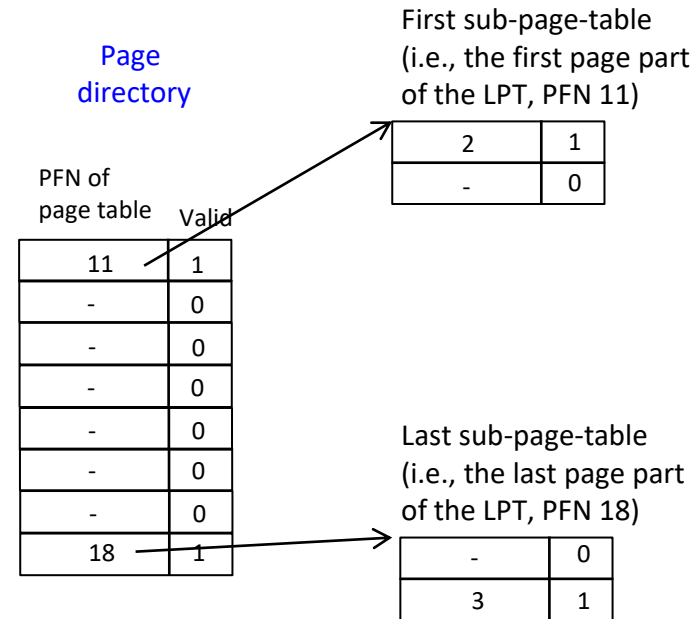
A multi-level paging example

Linear paging

	PFN	Valid	
PTE 0	2	1	1 st page of the LPT (PFN 11)
PTE 1	-	0	
PTE 2	-	0	2 nd page of the LPT (PFN 12)
PTE 3	-	0	
PTE 4	-	0	3 rd page of the LPT (PFN 13)
PTE 5	-	0	
PTE 6	-	0	4 th page of the LPT (PFN 14)
PTE 7	-	0	
PTE 8	-	0	5 th page of the LPT (PFN 15)
PTE 9	-	0	
PTE 10	-	0	6 th page of the LPT (PFN 16)
PTE 11	-	0	
PTE 12	-	0	7 th page of the LPT (PFN 17)
PTE 13	-	0	
PTE 14	-	0	8 th page of the LPT (PFN 18)
PTE 15	3	1	

8 pages are needed to store the address translation info

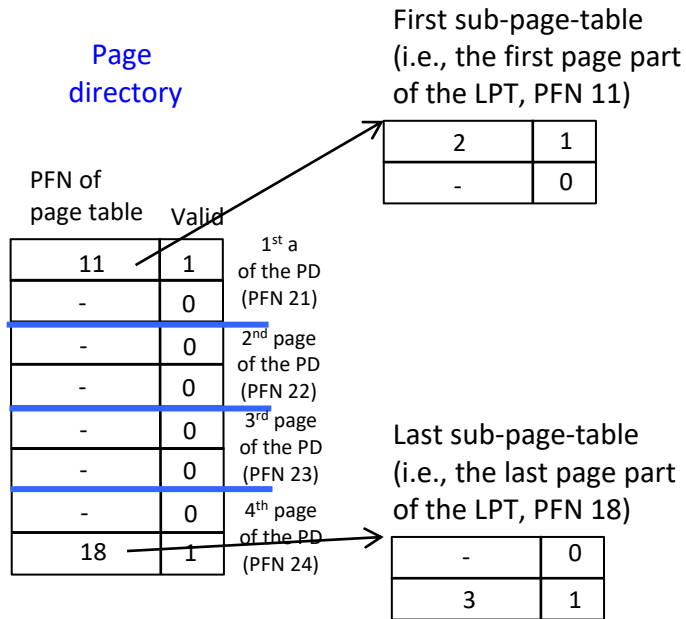
Two-level paging



Suppose PDE is the same size as PTE (i.e., 4 bytes), 6 pages are needed to store the address translation info

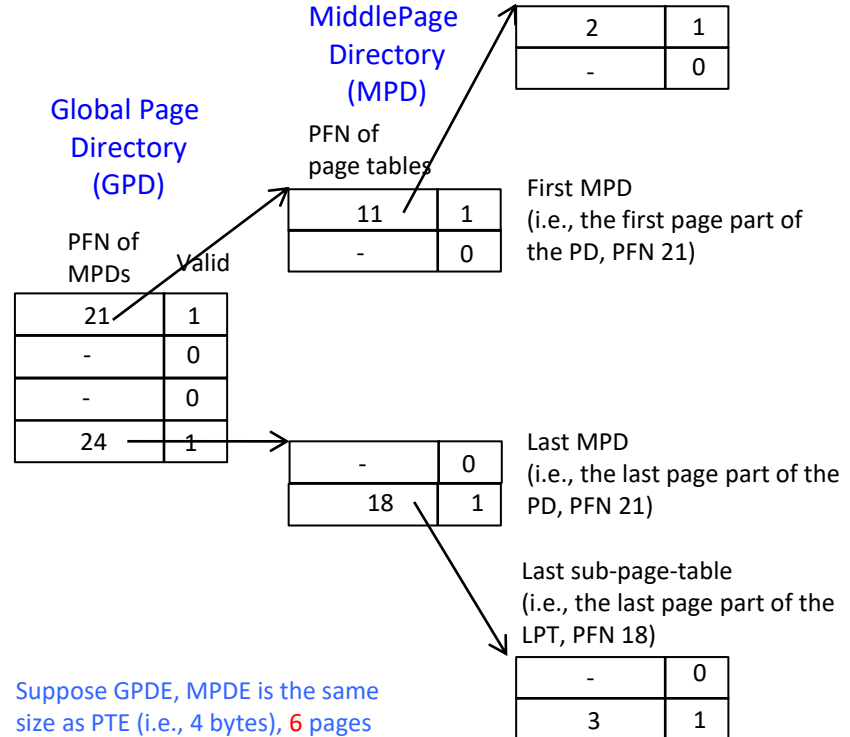
A multi-level paging example

Two-level paging



Suppose PDE is the same size as PTE (i.e., 4 bytes), 6 pages are needed to store the address translation info

Three-level paging



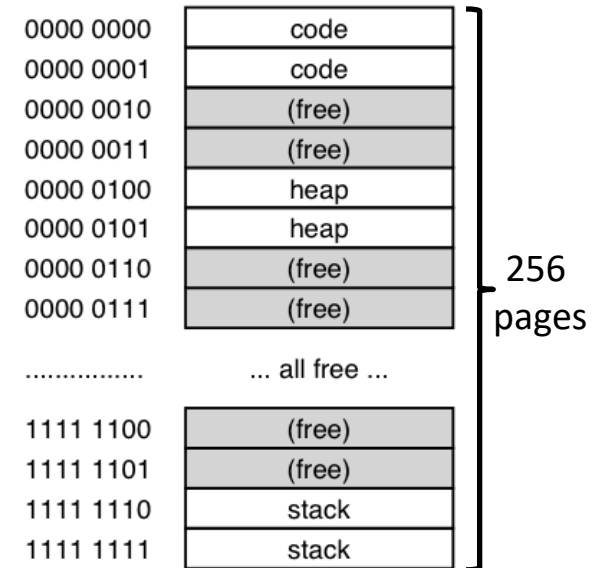
Suppose GPDE, MPDE is the same size as PTE (i.e., 4 bytes), 6 pages are needed to store the address translation info.

- No saving, just because the AS is small.
- In reality 3 or 4 level paging is commonly used

Another multi-level paging example

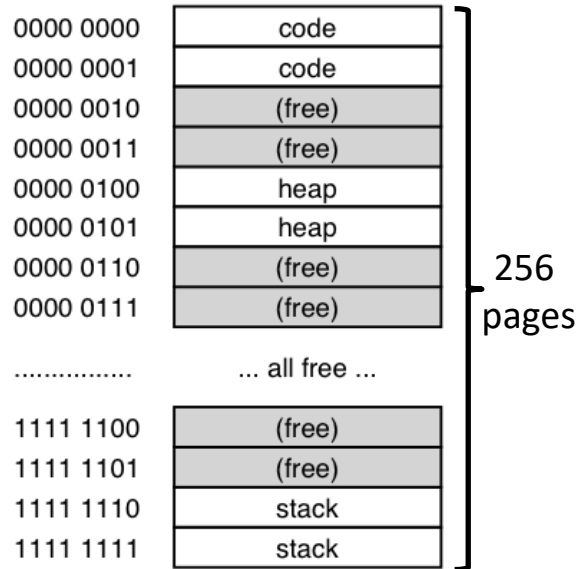
- Basic setup
 - 16 KB address space
 - Page size is 64 bytes
 - PTE size is 4 bytes
- Q1: How many bits for VA, VPN, offset respectively?
 - VA: 14 bits ($2^{14} = 16 \text{ KB}$)
 - VPN: 8 bits (there are $16 \text{ KB} / 64 = 2^{14} / 2^6 = 2^8 = 256$ pages)
 - Offset: 6 bits ($2^6 = 64$)
- Q2: Suppose linear paging is used, how many pages are needed to store a linear page table (LPT)?
 - Since there are 2^8 pages, the linear page table has 2^8 PTEs.
 - Therefore, the size of a linear page table is $2^8 \times 4 = 2^{10}$ bytes, which occupy $2^{10} / 64 = 2^{10} / 2^6 = 2^4 = 16$ pages.
- Q3: The address space of a process P looks like something on the right. How many pages of physical memory does a two-level paging approach (with the LPT divided into page-sized units) save compared to using linear paging?

Address space of a process P

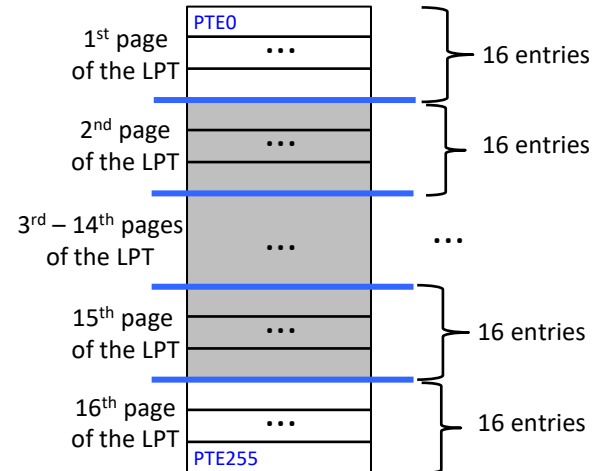


Another multi-level paging example

Suppose the address space of a process P at a certain time looks like:

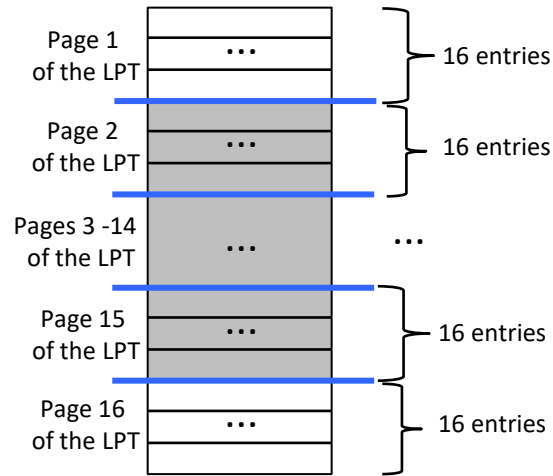


Then the **linear page table (LPT)**, which occupies 16 pages, looks like:



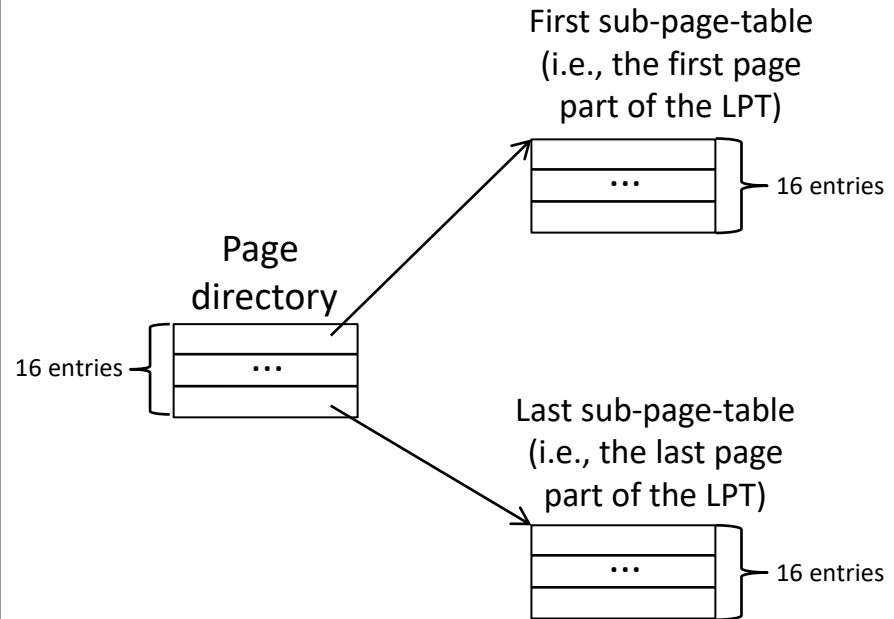
Another multi-level paging example

Linear paging



16 pages are needed to store the address translation info

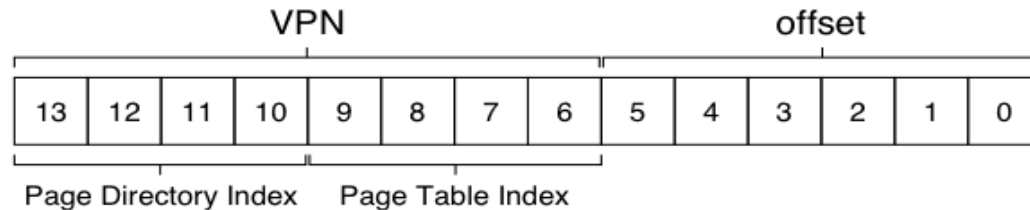
2-level paging



Suppose PDE is the same size as PTE (i.e., 4 bytes), 3 pages are needed to store the address translation info

Another multi-level paging example

- With linear paging, a virtual address has 14 bits (8 bit VPN, 6 bit offset)
- With two-level paging shown previously, the 8 bit VPN can be broken down into:
 - 4 bit for **page directory index (PDI)** (because there are 16 entries in page directory)
 - 4 bit for **page table index (PTI)** (because each sub-page table have 16 PTEs)



See more details in the textbook

Multi-level Page Tables

- Pros
 - page-table space allocated is proportional to amount of address space used
 - growing page table by getting more page frames (which could be anywhere in physical memory)
- Cons?
 - on TLB miss, need two or more memory accesses to get translation info (one for page directory and one for PTE itself)
 - **time-space trade-off**

Inverted Page Table

- Instead of having many page tables (one per process), a single page table that has an entry for each *physical page* of the system
 - which process is using this page (frame)
 - which virtual page of that process maps to this physical page
- More extreme space savings
- PowerPC